

PC4

1. 本章位置：第二篇“并行算法的设计”

这份 **PC4.pdf** 是《并行计算》课程第二篇的开头，主题是 **第四章：并行算法的设计基础**。它主要讲两个问题：第一，什么是并行算法，怎样描述和度量并行算法；第二，常见并行计算模型有哪些，包括 PRAM、APRAM、BSP 和 LogP。

1.1 第二篇的整体结构

第二篇叫 **并行算法的设计**，包括四章：

- 第四章：并行算法的设计基础
- 第五章：并行算法的一般设计方法
- 第六章：并行算法的基本设计技术
- 第七章：并行算法的一般设计过程

这份课件只讲第四章。

1.2 第四章内容结构

第四章分为两节：

- **4.1 并行算法的基础知识**
- **4.2 并行计算模型**

其中 4.1 讲并行算法的定义、表达、复杂度、同步与通信；4.2 讲并行算法分析中常用的抽象机器模型。

2. 4.1 并行算法的基础知识

2.1 并行算法的定义

课件中对并行算法的定义是：

并行算法是一些可同时执行的进程的集合，这些进程相互作用和协调动作，从而达到给定问题的求解。

这句话有三个关键词：

2.1.1 可同时执行

并行算法不是单条执行路径，而是多个计算活动可以同时进行。

例如数组求和：

$$a_0 + a_1 + a_2 + \dots + a_{n-1}$$

串行算法是一个一个加；并行算法可以先两两相加，再把部分和继续合并。

2.1.2 互相作用

并行进程之间不是完全独立的。它们可能需要交换数据。

例如矩阵向量乘法中，每个处理器计算一部分结果，但最终可能需要把结果汇总到某个处理器。

2.1.3 协调动作

多个并行进程必须在某些时刻同步或通信，否则结果可能错误。

例如多个处理器同时更新同一个变量 `S`，如果没有加锁，就可能出现数据竞争。

3. 并行算法的分类

课件给出四种分类方式。

3.1 数值计算算法和非数值计算算法

3.1.1 数值计算算法

数值计算算法主要处理连续数学问题，常见于科学计算和工程计算。

例如：

- 矩阵乘法
- 线性方程组求解
- 快速傅里叶变换
- 微分方程数值求解
- 流体力学模拟

这类问题通常有大量浮点运算，适合高性能计算机。

3.1.2 非数值计算算法

非数值计算算法主要处理离散结构。

例如：

- 图搜索
- 排序
- 字符串匹配
- 数据库查询
- 组合优化
- 符号计算

这类问题往往通信和控制更复杂，不一定像矩阵运算那样规则。

3.2 同步算法和异步算法

3.2.1 同步算法

同步算法中，各处理器按照统一节奏推进。

例如：

```
第 1 步：所有处理器计算局部结果
同步
第 2 步：所有处理器交换数据
同步
第 3 步：所有处理器继续计算
```

优点是容易分析，缺点是慢处理器会拖住所有处理器。

3.2.2 异步算法

异步算法中，各处理器不必等待统一时钟，可以按自己的速度执行。

优点是更灵活，可能减少等待；缺点是正确性分析更困难。

3.3 分布算法

分布算法是指运行在分布式系统中的算法。每个处理器通常有自己的局部存储，进程之间通过消息通信。

它强调：

- 没有全局共享内存；
- 节点之间通过 `send/receive` 通信；
- 通信延迟和消息顺序会影响算法设计；
- 容错、同步、选主、共识等问题很重要。

这类算法和后面讲的 MIMD-DM、BSP、LogP 模型关系密切。

3.4 确定算法和随机算法

3.4.1 确定算法

同样输入下，每次执行都得到同样的计算路径和结果。

例如普通的并行归约求和，如果调度和浮点误差不考虑，则属于确定算法。

3.4.2 随机算法

算法执行过程中使用随机选择。

例如随机快速排序、随机负载均衡、随机图算法等。

随机算法常用于打破对称性、降低最坏情况复杂度，或者简化并行冲突处理。

4. 并行算法的表达

并行算法需要一种表达方式，让人能清楚看出哪些语句可以并行执行。

4.1 描述语言

课件说可以使用类 Algol、类 Pascal 等描述语言，并在其中引入并行语句。

也就是说，并行算法的伪代码通常不是直接写某种真实编程语言，而是用结构化伪代码表达算法思想。

4.2 Par-do 语句

`par-do` 表示循环中的多个迭代可以并行执行。

课件给出的形式是：

```
for i = 1 to n par-do
  ...
end for
```

含义是：

`i = 1, 2, ..., n` 这些循环迭代可以同时执行。

例如数组加法：

```
for i = 1 to n par-do
  C[i] = A[i] + B[i]
end for
```

每个 `C[i]` 的计算互不依赖，所以可以并行。

4.3 for all 语句

`for all` 表示对一组处理器或一组对象同时执行某操作。

课件形式是：

```
for all Pi, where 0 ≤ i ≤ k
  ...
end for
```

含义是：

所有满足条件的处理器 `Pi` 同时执行相应代码。

例如：

```
for all Pi, where 0 ≤ i ≤ p-1
  Pi computes local_sum[i]
end for
```

这表示所有处理器同时计算自己的局部和。

5. 并行算法的复杂性度量

并行算法不能只看时间复杂度，还要看处理器数、通信代价、总工作量和成本。

5.1 串行算法复杂度度量

课件先回顾串行算法复杂性。

5.1.1 最坏情况复杂度

Worst-case complexity 表示在所有输入中最坏输入对应的运行时间。

例如排序算法的最坏情况复杂度可能是：

$$O(n \log n)$$

5.1.2 期望复杂度

Expected complexity 表示在某种概率分布下的平均运行时间。

随机算法中经常使用期望复杂度。

5.2 并行算法的运行时间 $t(n)$

并行算法运行时间 $t(n)$ 包含：

- 计算时间
- 通信时间

其中 n 是问题输入规模。

和串行算法不同，并行算法中通信时间可能非常重要。
有些算法计算量很小，但通信频繁，最终性能反而不好。

5.3 处理器数 $p(n)$

$p(n)$ 表示并行算法使用的处理器数量。

一个算法即使时间很短，如果需要极多处理器，也不一定实际可行。

例如：

$$\text{时间 } O(1), \text{ 处理器 } O(n^2)$$

这种算法理论上很快，但硬件成本可能很高。

5.4 并行算法成本 $c(n)$

课件定义：

$$c(n) = t(n) p(n)$$

即：

$$\text{并行算法成本} = \text{运行时间} \times \text{处理器数}$$

这个指标可以理解为总资源消耗。

如果一个并行算法使用 p 个处理器运行 t 时间，那么它总共消耗了约 pt 个处理器时间单位。

5.5 总运算量 $W(n)$

$w(n)$ 是并行算法求解问题时完成的总操作步数。

它和成本 $c(n)$ 有关系，但不完全一样。

例如有些处理器可能在等待同步或通信，这些等待会计入成本，但不一定计入有效运算量。

6. Brent 定理

课件第 10 页给出 Brent 定理。

6.1 定理内容

设：

- $w(n)$ 是并行算法 A 的总运算量；
- $T(n)$ 是该算法在无限多处理器或足够多处理器下的并行时间，也可理解为关键路径长度；
- 使用 p 台处理器执行。

则算法可在：

$$t(n) = O(w(n)/p + T(n))$$

时间内执行完毕。

6.2 直观理解

这个公式包含两部分：

$$w(n)/p$$

表示总工作量平均分给 p 个处理器后的理想计算时间。

$$T(n)$$

表示算法本身的依赖链长度，即无论处理器再多也无法突破的时间下限。

所以 Brent 定理告诉我们：

并行运行时间既受总工作量限制，也受依赖深度限制。

6.3 例子理解

假设一个并行算法总共要做 $w = 1000$ 次操作，关键路径长度 $T = 20$ 。

如果使用 $p = 10$ 个处理器，则时间大约受：

$$w/p + T = 1000/10 + 20 = 120$$

控制。

如果使用 $p = 100$ 个处理器：

$$w/p + T = 1000/100 + 20 = 30$$

但即使用无限多处理器，也不能小于 $T = 20$ 。

这说明处理器数量增加到一定程度后，加速会变慢。

6.4 $W(n)$ 和 $c(n)$ 的关系

课件指出：

当：

$$p = O(w(n)/T(n))$$

时， $w(n)$ 和 $c(n)$ 渐进一致。

并且对于任意 p ：

$$c(n) \geq w(n)$$

意思是：

并行算法成本至少不能小于实际总运算量，因为等待、同步和通信只会增加资源消耗。

7. 并行算法中的同步

7.1 同步概念

课件定义同步为：

在时间上强使各执行进程在某一点必须互相等待。

同步可以通过：

- 软件
- 硬件
- 固件

实现。

常见同步方式包括：

- barrier
- lock
- semaphore
- event
- condition variable

7.2 为什么需要同步

并行算法中多个进程可能同时访问共享资源。

如果没有同步，可能出现：

- 数据竞争
- 写冲突
- 读到旧值
- 结果不确定
- 死锁或活锁

例如多个处理器同时执行：

```
S = S + L
```

如果没有锁，两个处理器可能同时读取旧的 S ，然后分别写回，导致其中一个加法结果丢失。

7.3 共享存储多处理器求和算法

课件给出算法 4.1：共享存储多处理器上求和算法。

输入：

```
A = (a0, ..., an-1)
处理器数 p
```

输出：

```
S =  $\sum a_i$ 
```

算法思想如下：

```
Begin
  S = 0

  for all  $P_i$  where  $0 \leq i \leq p-1$  do
    L = 0

    for j = i to n step p do
      L = L +  $a_j$ 
    end for

    lock(S)
    S = S + L
    unlock(S)
  end for
End
```

7.4 算法解释

每个处理器 P_i 负责一部分数组元素：

```
Pi 处理  $i, i+p, i+2p, \dots$ 
```

先把这些元素加到局部变量 L 中。

然后所有处理器需要把自己的 L 加到全局共享变量 S 上。

因为 S 是共享变量，所以更新 S 时必须加锁：

```
lock(S)
S = S + L
unlock(S)
```

这段是临界区。

7.5 这个算法的特点

优点：

- 简单直观；
- 每个处理器工作量大致均衡；
- 局部求和阶段并行度高。

缺点：

- 最后更新 S 时需要互斥锁；
- 如果处理器很多，锁竞争会成为瓶颈；
- 更高效的做法通常是树形归约，而不是所有处理器争抢同一个锁。

8. 并行算法中的通信

8.1 通信概念

并行进程之间需要交换信息。

不同体系结构下通信方式不同。

8.2 共享存储多处理器通信

共享存储多处理器使用共享变量通信。

课件给出两个抽象操作：

```
global read(X, Y)
global write(X, Y)
```

可以理解为：

- `global read(X, Y)`：从共享变量 X 读到本地变量 Y ；
- `global write(X, Y)`：把本地或某个值写到共享变量 X 。

这种模式下，通信是隐式的，本质是读写共享内存。

8.3 分布存储多计算机通信

分布存储系统没有共享内存，通信需要显式消息传递。

课件给出两个操作：

```
send(X, i)
receive(Y, j)
```

含义是：

- `send(X, i)`：把 X 发送给处理器 i ；
- `receive(Y, j)`：从处理器 j 接收消息到 Y 。

这种模式下，程序员必须明确指定通信对象和通信内容。

8.4 分布存储多计算机上矩阵向量乘算法

课件给出算法 4.2：分布存储多计算机上的矩阵向量乘算法。

问题是计算：

$$y = A x$$

矩阵 A 和向量 x 被划分到不同处理器。

每个处理器 P_i 持有：

$$\begin{aligned} B &= A[1..n, (i-1)r+1 .. ir] \\ w &= x[(i-1)r+1 .. ir] \end{aligned}$$

也就是说，矩阵按列块划分，向量也按对应分块划分。

每个处理器先计算局部乘积：

$$z = B w$$

然后所有处理器的局部结果需要累加起来，得到完整的 Ax 。

8.5 环式累加通信过程

课件算法大致是：

```
Begin
  Compute z = B w

  if i = 1 then
    y = 0
  else
    receive(y, left)
  endif

  y = y + z

  send(y, right)

  if i = 1 then
    receive(y, left)
  endif
End
```

这里可以理解为处理器构成一个环：

$$P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow \dots \rightarrow P_p \rightarrow P_1$$

每个处理器收到前一个处理器传来的部分和 y ，加上自己的局部结果 z ，再传给下一个处理器。

最后结果回到 P_1 ，由 P_1 保存完整的 Ax 。

8.6 这个算法体现的问题

它说明分布存储算法必须考虑：

- 数据如何划分；
- 每个处理器计算什么；
- 局部结果如何合并；
- 通信路径如何安排；
- 谁保存最终结果。

这比共享存储模型更复杂，但更适合大规模系统。

9. 4.2 并行计算模型

并行计算模型是为了分析并行算法而抽象出来的机器模型。

本节讲四个模型：

- PRAM

- 异步 APRAM
- BSP
- LogP

这些模型从理想到现实逐步过渡。

10. PRAM 模型

10.1 基本概念

PRAM 是 **Parallel Random Access Machine**，并行随机访问机。

课件说 PRAM 由 Fortune 和 Wyllie 于 1978 年提出，又称 **SIMD-SM 模型**。

其中：

- SIMD：单指令流多数据流；
- SM：Shared Memory，共享存储。

PRAM 有：

- 一个集中共享存储器；
- 一个指令控制器；
- 多个处理器；
- 每个处理器可有局部存储器；
- 处理器通过共享存储器读写交换数据；
- 计算是隐式同步的。

10.2 PRAM 结构

结构可以理解为：

```
Control Unit
|
Processors + Local Memories
|
Interconnection Network
|
Shared Memory
```

即所有处理器通过互连网络访问同一个共享存储器。

11. PRAM 的分类

PRAM 的分类取决于多个处理器是否可以同时读写同一个共享存储单元。

11.1 CRCW PRAM

CRCW 是：

```
Concurrent Read Concurrent Write
```

即并发读、并发写。

多个处理器可以同时读同一个存储单元，也可以同时写同一个存储单元。

但是同时写会产生冲突，所以又分三种。

11.1.1 Common CRCW

Common PRAM-CRCW 只允许多个处理器写入相同数据。

例如多个处理器同时写：

```
x = 1
```

这是允许的。

如果有人写 1，有人写 2，则不允许。

11.1.2 Priority CRCW

Priority PRAM-CRCW 只允许优先级最高的处理器写入成功。

例如 P0, P1, P2 同时写 X, 如果 P0 优先级最高, 则 P0 的值生效。

11.1.3 Arbitrary CRCW

Arbitrary PRAM-CRCW 允许任意一个处理器写入成功。

如果多个处理器同时写, 模型不规定到底谁成功, 只保证其中一个成功。

11.2 CREW PRAM

CREW 是:

Concurrent Read Exclusive Write

即并发读、互斥写。

多个处理器可以同时读同一个存储单元, 但写同一个存储单元时必须互斥。

这是比 CRCW 更弱、更现实一点的模型。

11.3 EREW PRAM

EREW 是:

Exclusive Read Exclusive Write

即互斥读、互斥写。

任何时刻, 同一个存储单元不能被多个处理器同时读, 也不能被多个处理器同时写。

这是最严格、最弱的 PRAM 模型。

12. PRAM 模型计算能力比较

12.1 三种 PRAM 强弱关系

课件给出:

$T_{EREW} \geq T_{CREW} \geq T_{CRCW}$

意思是:

- CRCW 最强;
- CREW 次之;
- EREW 最弱。

同一个问题在 CRCW 上可能更快, 因为它允许更强的并发读写。

12.2 EREW 对其他模型的模拟

课件指出, PRAM-EREW 可以用 $\log p$ 倍时间模拟 CREW 和 CRCW。

即:

$T_{EREW} = O(T_{CREW} \log p)$

以及:

$T_{EREW} = O(T_{CRCW} \log p)$

含义是:

虽然 EREW 限制更强, 但通过额外的树形广播、归约等方法, 可以模拟更强模型, 只是要付出对数因子代价。

13. PRAM 模型优缺点

13.1 优点

PRAM 适合：

- 并行算法表示；
- 并行复杂性分析；
- 理论研究；
- 细粒度并行算法设计。

它隐藏了：

- 通信细节；
- 同步细节；
- 网络拓扑；
- 访存延迟；
- 存储竞争。

所以使用起来很简单。

13.2 缺点

PRAM 不适合真实 MIMD 并行机。

主要原因是它忽略了：

- 共享存储器竞争；
- 通信延迟；
- 同步开销；
- 网络带宽限制；
- Cache 一致性；
- 存储访问冲突。

所以 PRAM 更像理论模型，不是现实机器模型。

14. 异步 APRAM 模型

14.1 基本概念

APRAM 是 Asynchronous PRAM，异步 PRAM。

课件也称它为：

分相 Phase PRAM

或：

MIMD-SM 模型

与 PRAM 不同，APRAM 中每个处理器有：

- 局部存储器；
- 局部时钟；
- 局部程序。

系统没有全局时钟，各处理器异步执行。

处理器之间仍然通过共享存储器 SM 通信。

如果处理器之间存在依赖关系，需要在并行程序中显式加入同步路障。

14.2 APRAM 指令类型

课件列出四类指令：

14.2.1 全局读

从共享存储器读取数据。

14.2.2 全局写

向共享存储器写入数据。

14.2.3 局部操作

处理器在本地存储器或本地寄存器上执行计算。

14.2.4 同步

通过同步障或路障协调多个处理器的执行阶段。

15. APRAM 的计算过程

15.1 全局相 Phase

APRAM 的计算由多个全局相组成。

相与相之间用同步障隔开。

结构类似：

```
Phase 1:
    各处理器异步执行局部操作、全局读写
Barrier

Phase 2:
    各处理器异步执行下一阶段操作
Barrier

Phase 3:
    ...
```

15.2 与 PRAM 的区别

PRAM 是每一步所有处理器隐式同步。

APRAM 是每个 phase 内异步执行，phase 之间显式同步。

所以 APRAM 比 PRAM 更接近真实共享内存并行程序。

16. APRAM 的计算时间

16.1 参数设定

课件设定：

- 局部操作为单位时间；
- 全局读/写平均时间为 d ；
- d 随处理器数增加而增加；
- 同步路障时间为 $B = B(p)$ ，是非降函数。

课件给出关系：

$$2 \leq d \leq B \leq p$$

并且：

$$B = B(p) = O(d \log p)$$

或：

$$O(d \log p / \log d)$$

16.2 APRAM 时间公式

令 t_{ph} 为一个全局相内各处理器执行时间最长者，则 APRAM 上的计算时间为：

$$T = \sum t_{ph} + B \times \text{同步障次数}$$

16.3 公式含义

每个 phase 的时间由最慢处理器决定。

此外，每次 phase 结束都有 barrier，因此同步次数越多，总时间越大。

这说明在 APRAM 上设计并行算法时，要尽量减少不必要的同步障。

17. APRAM 模型优缺点

17.1 优点

APRAM 相比 PRAM 更现实，因为它允许：

- 处理器异步运行；
- 显式同步；
- 局部程序；
- 局部时钟；
- 更接近 MIMD-SM。

它仍然比较容易编程和分析复杂度。

17.2 缺点

课件指出它与现实仍有较大差距。

主要原因：

- 仍然假设共享存储；
- 不适合分布存储 MIMD-DM；
- 可在其上设计的并行算法有限；
- 通信和存储竞争仍然被抽象得较粗。

18. BSP 模型

18.1 基本概念

BSP 是 **Bulk Synchronous Parallel**，块同步并行模型。

课件说它由 Valiant 于 1990 年提出。

它是一种：

异步 MIMD-DM 模型

其中：

- MIMD：多指令流多数据流；
- DM：Distributed Memory，分布存储。

BSP 支持消息传递系统。

它的特点是：

块内异步并行，块间显式同步

18.2 BSP 模型参数

BSP 有三个核心参数。

18.2.1 p: 处理器数

p 表示处理器数量，每个处理器带有自己的存储器。

18.2.2 l: 同步障时间

l 是 barrier synchronization time，即一次全局同步所需的时间。

18.2.3 g: 带宽因子

g 表示：

time steps / packet

也就是传输一个数据包所需的时间步数。

课件写作：

$g = 1 / \text{bandwidth}$

带宽越高，g 越小。

19. BSP 的计算过程

19.1 超级步 Superstep

BSP 计算由若干超级步组成。

每个超级步包括：

局部计算 → 全局通信 → 路障同步

课件第 25 页图 4.3 展示了这一模式：上方是各处理器进行局部计算，中间有全局通信，最后统一进入路障同步。

19.2 局部计算

每个处理器先在自己的数据上执行计算，不需要和其他处理器同步。

19.3 全局通信

处理器之间通过消息传递交换数据。

19.4 路障同步

所有处理器在超级步末尾同步，确保这一轮通信完成后再进入下一轮。

20. BSP 模型优缺点

20.1 优点

BSP 强调计算和通信分离。

这带来几个好处：

- 程序结构清晰；
- 容易分析复杂度；
- 容易进行性能预测；
- 适合消息传递系统；
- 提供较好的编程抽象。

BSP 可以看作并行算法设计中的“结构化编程模型”。

20.2 缺点

课件指出：

- 需要显式同步机制；
- 每个超级步中消息传递受到限制，最多传递 h 条消息；
- 对某些异步细粒度程序不够灵活；
- 频繁 barrier 会增加开销。

21. LogP 模型

21.1 基本概念

LogP 模型由 Culler 于 1993 年提出。

它是一种：

分布存储、点到点通信的多处理机模型

通信由一组参数描述，实行隐式同步。

LogP 比 BSP 更关注真实机器中的通信代价。

21.2 LogP 四个参数

LogP 的名字来自四个参数。

21.2.1 L: network latency

L 表示网络延迟。

也就是消息从发送端到接收端经历的最大或典型延迟。

21.2.2 o: communication overhead

o 表示通信开销。

它是处理器参与发送或接收消息时不能用于其他计算的时间。

21.2.3 g: gap

g 表示连续发送或接收消息之间的最小间隔。

课件写作：

$g = 1 / \text{bandwidth}$

g 反映网络带宽限制。

21.2.4 P: processors

P 表示处理器数量。

21.3 L 和 g 的含义

课件特别说明：

L 和 **g** 反映了通信网络的容量

其中：

- **L** 反映消息延迟；
- **g** 反映消息吞吐能力。

22. LogP 模型优缺点

22.1 优点

LogP 捕捉了多处理机通信瓶颈。

它隐藏了：

- 网络拓扑；
- 路由算法；
- 通信协议。

同时又比 PRAM、APRAM 更关注实际通信代价。

它可以用于：

- 共享存储编程模型；
- 消息传递编程模型；
- 数据并行编程模型。

22.2 缺点

LogP 难以用于算法描述、设计和分析。

因为参数更多，通信行为更细，算法设计者需要考虑更多底层因素。

所以 LogP 更适合性能建模，而 BSP 更适合结构化算法设计。

23. BSP 与 LogP 的关系

23.1 从 BSP 到 LogP

课件给出：

BSP 块同步 $\rightarrow$ BSP 子集同步 $\rightarrow$ BSP 进程对同步 = LogP

意思是：

- BSP 要求所有处理器在超级步末尾同步；
- 如果放宽为部分处理器同步，就变成子集同步；
- 如果进一步变成进程对之间同步，就更接近 LogP 的点到点通信模型。

23.2 相互模拟关系

课件指出：

BSP 可以常数因子模拟 LogP

而：

LogP 可以对数因子模拟 BSP

说明两者表达能力接近，但模拟代价不同。

23.3 关系式

课件给出一个简洁关系：

$BSP = LogP + Barriers - Overhead$

可以理解为：

- BSP 在 LogP 的点到点通信基础上加入了 barrier；
- 同时把许多底层通信开销抽象掉。

23.4 选择 BSP 还是 LogP

课件结论是：

- BSP 提供更方便的程序设计环境；
- LogP 能更好地利用机器资源；
- BSP 更简单、方便，也更符合结构化编程。

所以：

做算法设计：BSP 更合适
做性能建模：LogP 更细致

24. 本章总结

24.1 本章讲了什么

PC4.pdf 主要讲 并行算法设计基础，包括：

并行算法基础知识

- 定义和分类
- 表达方式
- 复杂性度量
- 同步
- 通信

并行计算模型

- PRAM
- APRAM
- BSP
- LogP

24.2 核心理解

这一章的核心思想是：

并行算法不仅要考虑计算步骤，还必须考虑处理器数量、通信、同步、存储模型和机器抽象。

串行算法主要关注：

时间复杂度

空间复杂度

并行算法还要额外关注：

处理器数

并行时间

总工作量

成本

通信代价

同步代价

模型适配性

24.3 四种模型的对比

模型	存储	同步	通信	适合用途
PRAM	集中共享存储	隐式同步	共享存储读写	理论算法分析
APRAM	共享存储	显式 barrier	全局读写	异步共享存储算法
BSP	分布存储	超级步 barrier	消息传递	结构化并行算法设计
LogP	分布存储	隐式同步	点到点消息	通信性能建模

24.4 复习重点

这份课件最应该掌握：

- 并行算法的定义；
- 并行算法的分类；
- par-do 和 for all 的含义；
- 并行时间 $t(n)$ 、处理器数 $p(n)$ 、成本 $c(n)$ 、总运算量 $w(n)$ ；
- Brent 定理：

$$t(n) = O(w(n)/p + T(n))$$

- 同步与通信的区别；
- PRAM 的 CRCW、CREW、EREW；
- APRAM 的 phase 和 barrier；
- BSP 的 superstep；
- LogP 的四个参数：

$$L, o, g, P$$

整章可以一句话概括：

并行算法设计的第一步，是选定合适的计算模型，并在该模型下准确描述算法、分析时间、处理器数、通信和同步开销。

PC1-3.pdf

1. 课件总体结构

这份课件先给出整本课程的目录。全书分为四篇：

第一篇：并行计算的基础

包括：

- 第一章：并行计算机系统及其结构模型
- 第二章：当代并行机系统：SMP、MPP 和 Cluster
- 第三章：并行计算性能评测

这正是本文件主要讲解的内容。

第二篇：并行算法的设计

包括并行算法设计基础、设计方法、基本设计技术和一般设计过程。

第三篇：并行数值算法

包括基本通信操作、稠密矩阵运算、线性方程组求解、快速傅里叶变换。

第四篇：并行程序设计

包括并行程序设计基础、共享存储系统编程、分布存储系统编程、并行程序设计环境与工具。

这份 PC1-3 主要对应第一篇，所以重点是 **机器结构、互连网络、存储模型、当代并行机和性能评测**。

2. 第一章：并行计算机系统及结构模型

第一章分三大部分：

2.1 并行计算

并行计算指在并行机上进行的计算，也常被称为 **高性能计算** 或 **超级计算**。

它服务于计算科学，例如：

- 计算物理
- 计算化学
- 计算生物
- 计算材料
- 计算流体力学

课件列举的典型科学与工程需求包括：

- 气象预报
- 油藏模拟
- 核武器数值模拟
- 航天器设计
- 基因测序

这些问题的共同特点是规模巨大，普通单机难以承担。

2.2 三类计算需求

课件把科学工程问题的需求分成三类：

2.2.1 计算密集

主要瓶颈是计算量大。

例如核武器模拟、气候模拟、流体力学仿真。
这类问题需要大量浮点运算能力。

2.2.2 数据密集

主要瓶颈是数据量大。

例如基因测序、大规模数据库、遥感图像处理。
这类问题要求高存储容量、高 I/O 带宽和高数据吞吐能力。

2.2.3 网络密集

主要瓶颈是通信和互连。

例如分布式协同计算、网格计算、大规模集群计算。
这类问题不仅要算得快，还要求节点之间数据交换快。

2.3 高性能计算机发展例子

课件列举了若干早期高性能计算系统：

- Intel Option Red：1997 年，1 Tflops，Pentium Pro
- SGI Option Blue Mountain：1998 年，3 Tflops，MIPS R10000
- IBM Option White：2001 年，7 Tflops，Power3
- 日本 Earth Simulator：2002 年，35 Tflops，Top1，向量处理器
- HP ASCI Q：2002 年，7 Tflops，Alpha Server
- 中国联想：2002 年，1 Tflops，Top43

这里想说明：高性能计算机的发展方向一直是 **更多处理器、更强互连、更大内存、更高 I/O 能力**。

3. 并行计算机系统互连

并行机的核心问题之一是：多个处理器、内存和 I/O 设备之间如何连接。

3.1 系统互连层次

课件把互连技术按距离和范围分成：

3.1.1 总线或开关

用于节点内部，距离最短。
例如处理器总线、存储器总线、I/O 总线。

3.1.2 SAN

System Area Network，系统区域网络。
用于节点之间互连，常见于高性能集群内部。

3.1.3 LAN

Local Area Network，局域网。
用于一个建筑、实验室或校园内部的系统连接。

3.1.4 MAN

Metropolitan Area Network，城域网。
用于城市范围内的网络连接。

3.1.5 WAN

Wide Area Network，广域网。
用于跨地区、跨国家的大范围互连。

课件第 7 页图把总线、SAN、LAN、MAN、WAN 和带宽范围放在一起比较：越靠近机器内部，通常带宽越高、距离越短；越靠近 WAN，距离越远，但带宽和延迟更受限制。

4. 网络性能指标

课件第 9 页定义了几个重要网络指标。

4.1 节点度 Node Degree

节点度是射入或射出一个节点的边数。

如果是单向网络，入边和出边之和称为节点度。

节点度越高，说明一个节点可以直接连接更多邻居，但硬件复杂度也更高。

4.2 网络直径 Network Diameter

网络直径是网络中任意两个节点之间的最长最短路径。

也就是最坏情况下，两个节点通信最多要经过多少跳。

网络直径越小，远距离通信延迟通常越低。

4.3 对剖宽度 Bisection Width

对剖宽度是把网络分成大小相近的两半时，必须移去的最少边数。

它反映网络整体通信能力。

对剖宽度越大，说明网络两半之间可并行传输的数据越多。

4.4 对剖带宽 Bisection Bandwidth

对剖带宽是在最小对剖平面上，所有连线每秒能通过的最大信息量。

它比对剖宽度更具体，因为考虑了每条链路的带宽。

4.5 对称性 Symmetry

如果从任意节点看网络结构都一样，则称网络是对称的。

对称网络通常更容易路由，也更容易实现负载均衡。

5. 静态互连网络与动态互连网络

5.1 静态互连网络

静态互连网络中，处理单元之间的连接是固定的。

程序执行期间，点到点链路不变。

典型静态网络包括：

- 一维线性阵列
- 环
- 二维网孔
- 树
- 超立方
- 立方环
- 洗牌交换网
- 蝶形网络

静态网络的特点是结构固定、实现简单，但灵活性较低。

5.2 动态互连网络

动态互连网络由交换开关构成，可以根据程序需求动态改变连接关系。

典型动态网络包括：

- 总线
- 交叉开关
- 多级互连网络 MIN

动态网络灵活性更强，但硬件复杂度通常更高。

6. 典型静态互连网络

6.1 一维线性阵列

一维线性阵列是最简单的互连方式。

每个节点只与左、右近邻相连，也称二近邻连接。

对于 N 个节点：

- 链路数：N - 1
- 内部节点度：2
- 网络直径：N - 1
- 对剖宽度：1

缺点是网络直径大，远端通信很慢。

6.2 环形网络

如果把线性阵列的首尾节点相连，就得到环。

环可以是单向环，也可以是双向环。

双向环的特点：

- 节点度：2
- 网络直径约为 $\lfloor N/2 \rfloor$
- 对剖宽度：2

- 对称性好

相比线性阵列，环降低了最坏通信距离。

6.3 二维网孔 2-D Mesh

二维网孔中，每个内部节点连接上、下、左、右四个邻居。

对于 $\sqrt{N} \times \sqrt{N}$ 网孔：

- 节点度：4
- 网络直径： $2(\sqrt{N} - 1)$
- 对剖宽度： $\sqrt{N}$
- 边界节点连接较少，所以不是完全对称

二维网孔适合映射二维科学计算问题，例如矩阵、图像、网格计算。

6.4 Illiac 网孔

Illiac 网孔是在普通二维网孔基础上加入某些环绕和蛇形连接。

其目的是缩短通信距离，提高网络连通性。

它的节点度仍为 4，但直径比普通 Mesh 更小。

6.5 二维环绕 2-D Torus

二维 Torus 是在 Mesh 的水平和垂直方向都加入环绕连接。

它的特点：

- 节点度恒为 4
- 网络直径比 Mesh 更小
- 对剖宽度更大
- 对称性更好

很多并行机采用 Torus 或类似拓扑，因为它在成本和通信性能之间较平衡。

6.6 二叉树

二叉树中，每个内部节点连接父节点和两个子节点。

特点：

- 节点度通常为 3
- 对剖宽度为 1
- 直径约为 $2(\lceil \log N \rceil - 1)$

缺点是根节点容易成为通信瓶颈。

6.7 星形网络

如果把树的节点度增大到极端，让中心节点连接所有其他节点，就得到星形网络。

特点：

- 网络直径为 2
- 中心节点负载极重
- 对中心节点依赖很大

它通信路径短，但中心节点是严重瓶颈。

6.8 胖树 Fat Tree

胖树是对普通树的改进。

从叶节点到根节点，通路逐渐变宽。

这样可以缓解根附近通信瓶颈。

胖树思想在现代数据中心网络和集群网络中也很重要。

6.9 超立方 Hypercube

n 维超立方有：

$$N = 2^n$$

个节点。

特点：

- 节点度：n
- 网络直径：n

- 对剖宽度：N/2
- 对称性好

超立方通信性能好，但节点度随规模增长，硬件连接复杂度也随之提高。

6.10 立方环 Cube-Connected Cycles

立方环是把超立方中每个顶点替换成一个环。

这样可以把节点度固定为 3，而不像超立方那样节点度为 n。

代价是路径可能变长。

7. 网络嵌入

网络嵌入指把一个网络的节点映射到另一个网络中。

7.1 膨胀系数 Dilation

课件用膨胀系数描述嵌入质量。

它表示：

被嵌入网络中的一条链路，在目标网络中最多需要多少条链路来模拟。

如果膨胀系数为 1，就称为 **完美嵌入**。

7.2 完美嵌入例子

课件给出两个例子：

- 环网可以完美嵌入到 2-D Torus 中。
- 超立方网可以完美嵌入到 2-D Torus 中。

第 16 页图展示了超立方节点按二进制编号映射到二维环绕网中的方式。

嵌入的意义在于：

如果某个算法天然适合一种拓扑，而实际机器是另一种拓扑，就需要研究是否能高效映射。

8. 动态互连网络

8.1 总线

总线包括 PCI、VME、Multics、Sbus、MicroChannel 等。

多处理机总线系统要解决的问题包括：

- 总线仲裁
- 中断处理
- 协议转换
- 快速同步
- Cache 一致性协议
- 分事务
- 总线桥
- 层次总线扩展

总线优点是结构简单、成本低；缺点是带宽共享，扩展性差。

8.2 交叉开关 Crossbar

交叉开关是单级交换网络。

它像电话交换机一样，通过交叉点开关建立输入端口和输出端口之间的动态连接。

优点：

- 可提供高带宽
- 多个不冲突连接可同时进行
- 适合处理器间通信或处理器—存储器访问

缺点：

- 硬件复杂度高
- n 个输入输出时复杂度接近 $O(n^2)$

8.3 多级互连网络 MIN

MIN 是 Multistage Interconnection Network。

它由多个交换级串联而成，例如 Omega 网络。

课件第 20 页展示了 8 输入 Omega 网络：

输入和输出编号为 000 到 111，中间有多个 2×2 交换模块。

MIN 的特点是：

- 比 Crossbar 成本低
- 比总线并发能力强
- 路由有一定限制
- 可能出现阻塞和冲突

8.4 动态网络比较

课件比较了总线、多级互连网络和交叉开关：

网络	硬件复杂度	每处理器带宽	特点
总线	$O(n + w)$	$O(wf/n)$ 到 $O(wf)$	简单但共享带宽
多级互连网络	$O((n \log n)w)$	$O(wf)$	成本和性能折中
交叉开关	$O(n^2w)$	$O(wf)$	高带宽但复杂度最高

其中 n 是节点规模， w 是数据宽度， f 是时钟频率。

9. 标准互连网络

9.1 Myrinet

Myrinet 是 Myricom 公司设计的千兆位包交换网络，用于构筑计算机机群。

特点：

- 支持任意拓扑，不限于规则网孔或开关网孔。
- 数据链路层支持可变长包格式。
- 每条链路有流控制和错误控制。
- 使用切通选路法。
- 使用定制可编程主机接口。
- 物理层使用全双工 SAN 链路。

课件还提到 Myrinet 主机接口使用 LANai 芯片，包含 Myrinet 接口、包接口、DMA 引擎和 SRAM。

9.2 Myrinet 连接 LAN/Cluster

第 24 页图展示了 Myrinet 连接的 LAN/Cluster。

它可以连接：

- 桌面主机
- 机箱内多计算机机群
- 多处理机机群
- 网络 RAM 和磁盘
- VME 单板

这说明 Myrinet 不只是单纯网络，而是面向高性能集群系统互连。

9.3 HiPPI

HiPPI 是高性能并行接口，由 Los Alamos 国家实验室在 1987 年提出。

用途：

- 大型机和超级计算机接口统一
- 短距离系统到系统连接
- 系统到外设的高速 I/O 通道

特点：

- 物理层和数据链路层由 ANSI X3T9.3 标准认可
- 单工点到点数据传输接口
- 速率可达 800 Mbps 到 1.6 Gbps
- 超级 HiPPI 可达更高带宽

课件第 26 页图展示了使用 HiPPI 通道和开关构筑 LAN 主干网，可连接超级计算机、文件服务器、工作站、存储服务器和大规模并行处理系统。

9.4 光纤通道 Fiber Channel

光纤通道 FC 是通道和网络标准的集成。

特点：

- 可作为共享介质，也可作为交换技术
- 支持点到点、仲裁环和交换光纤连接
- 拓扑灵活
- 操作速度从 100 Mbps 到 800 Mbps，并发展到 1、2、4 Gbps

它的重要价值在于为高速存储和高速局域网提供基础。

9.5 FDDI

FDDI 是 Fiber Distributed Data Interface，光纤分布式数据接口。

特点：

- 双向光纤令牌环
- 传输速率 100 到 200 Mbps
- 可连接大量设备
- 传统 FDDI 主要以异步方式工作

第 28 页图展示了双向 FDDI 环作为主干网，连接文件服务器、数据库服务器、计算机服务器、桌面计算机、以太网集线器和路由器。

9.6 ATM

ATM 是 Asynchronous Transfer Mode，异步传输模式。

特点：

- 独立于介质
- 将消息分成固定长度 53 字节的报元
- 基于报元交换
- 目标是把实时数据和突发数据统一到一种网络技术中
- 支持 25、51、155、622 Mbps 等速率

第 30 页展示香港大学 Pearl 机群，图中 ATM 开关连接多种服务器、工作站池和外部网络，是标准互连网络在机群中的应用案例。

9.7 以太网、快速以太网和千兆以太网

第 31 页表格比较三代以太网：

类型	引入年代	带宽	主要应用
10BaseT 以太网	1982	10 Mb/s	文件共享、打印机共享
100BaseT 快速以太网	1994	100 Mb/s	COW 计算、C/S、大型数据库
千兆以太网	1997	1 Gb/s	大型图像、多媒体、互联网、数据仓库

以太网的优势是成本低、标准成熟、生态强，因此在 Cluster 中非常常见。

10. 并行计算机结构模型

课件第 32 页集中展示五种结构模型：PVP、SMP、MPP、DSM、COW。

10.1 PVP

PVP 是 Parallel Vector Processor，并行向量处理机。

特点：

- 使用向量处理器 VP
- 通过交叉开关连接共享存储器 SM
- 适合大规模向量科学计算
- 代表机器如 Cray C-90、Cray T-90、银河 1 号

10.2 SMP

SMP 是 Symmetric Multiprocessor，对称多处理机。

特点：

- 商用处理器 P/C
- 总线或交叉开关连接
- 集中共享存储器
- UMA 访问模型

- 易编程，但扩展性有限

10.3 MPP

MPP 是 Massively Parallel Processor，大规模并行处理机。

特点：

- 节点由处理器、本地内存和 NIC 组成
- 使用定制网络
- 分布式非共享存储
- 通信通常采用消息传递
- 访存模型通常是 NORMA

10.4 DSM

DSM 是 Distributed Shared Memory，分布式共享存储。

特点：

- 物理内存分布在各节点
- 逻辑上提供共享地址空间
- 使用目录 DIR 维护一致性
- 通常对应 NUMA 或 CC-NUMA
- 编程比纯消息传递更接近共享内存

10.5 COW

COW 是 Cluster of Workstations，工作站机群。

特点：

- 每个节点是完整计算机
- 有自己的内存、磁盘和操作系统
- 通过商品网络连接
- 通常使用消息传递
- 性价比高，灵活性强

11. 并行计算机体系合一结构

第 33 页指出：SMP、MPP、DSM 和 COW 的结构正在趋于一致。

11.1 趋同原因

主要原因是：

- 都使用大量商用微处理器
- 都通过高速网络连接节点
- 节点内部都有处理器、Cache、局部存储、NIC、磁盘等部件

11.2 Shell 结构

课件提出 Shell 结构：

 用专门定制的 Shell 电路，把商用微处理器和节点其他部分连接起来。

这些部分包括：

- 板级 Cache
- 局部存储
- NIC
- Disk

优点是 CPU 升级时只需要更换 Shell，整体系统结构可以保持稳定。

11.3 三种共享方式

图中展示三种结构：

- 无共享
- 共享磁盘
- 共享存储

这说明现代并行系统可以在共享程度上形成连续谱，而不是绝对分开。

12. 五种结构特性比较

第 34 页表格比较 PVP、SMP、MPP、DSM、COW。

12.1 处理器类型

- PVP 使用专用定制处理器。
- SMP、MPP、DSM、COW 多使用商用处理器。

12.2 互连网络

- PVP：定制交叉开关
- SMP：总线或交叉开关
- MPP：定制网络
- DSM：定制网络
- COW：商品网络，例如以太网或 ATM

12.3 通信机制

- PVP、SMP、DSM：共享变量
- MPP、COW：消息传递

12.4 地址空间

- PVP、SMP、DSM：单地址空间
- MPP、COW：多地址空间

12.5 访存模型

- PVP、SMP：UMA
- MPP、COW：NORMA
- DSM：NUMA

这张表是本章重点，说明不同体系结构和编程模型之间的对应关系。

13. 并行计算机访存模型

13.1 UMA

UMA 是 Uniform Memory Access，均匀存储访问。

特点：

- 物理存储器被所有处理器均匀共享
- 所有处理器访问任意存储字时间相同
- 每个处理器可以有私有 Cache
- 外设也可以某种形式共享

UMA 通常对应 SMP。

13.2 NUMA

NUMA 是 Nonuniform Memory Access，非均匀存储访问。

特点：

- 共享存储器物理上分布在所有处理器中
- 所有本地存储器共同构成全局地址空间
- 访问本地存储器快
- 访问远程存储器慢
- 每个处理器可带私有 Cache

NUMA 改善了集中共享内存的扩展性，但增加了访存不均匀性。

13.3 COMA

COMA 是 Cache-Only Memory Access，全高速缓存存储访问。

特点：

- 节点中没有传统存储层次
- 全部 Cache 组成全局地址空间
- 使用分布式高速缓存目录进行远程 Cache 访问
- 数据运行时会迁移到使用它的地方

COMA 的优点是数据局部性好，缺点是实现复杂。

13.4 CC-NUMA

CC-NUMA 是 Coherent-Cache NUMA，高速缓存一致性 NUMA。

特点：

- 使用基于目录的 Cache 一致性协议
- 保留 SMP 易编程优点
- 改善传统 SMP 的可扩放性
- 实际上是分布共享存储的 DSM 多处理机系统
- 程序员不必显式分配数据到节点

CC-NUMA 是 DSM 思想的重要硬件实现形式。

13.5 NORMA

NORMA 是 No-Remote Memory Access，非远程存储访问。

特点：

- 所有存储器都是私有的
- 处理器不能直接访问远程存储器
- 需要通过消息传递交换数据
- MPP 和 COW 通常采用 NORMA

NORMA 的优势是扩展性强，缺点是编程复杂。

13.6 MIMD 存储结构总图

第 40 页把 MIMD 系统分成两大类：

- 多处理机：单地址空间，共享存储器
- 多计算机：多地址空间，非共享存储器

对应关系大致是：

UMA	→ PVP, SMP
NUMA	→ COMA, CC-NUMA, NCC-NUMA
NORMA	→ Cluster, MPP
DSM	→ 软件或硬件支持的分布共享存储

14. 第二章：当代并行机系统

第二章讲三类当代并行系统：

- SMP
- MPP
- Cluster / COW

15. 对称多处理机 SMP

15.1 SMP 基本结构

SMP 使用商用微处理器，通常带片上或片外 Cache。

结构特点：

- 基于总线或交叉开关连接
- 集中式共享存储
- UMA 结构
- 所有处理器地位对称

代表系统：

- SGI Power Challenge
- DEC Alpha Server
- Dawning 1

15.2 SMP 优点

SMP 的优点包括：

- 对称性好
- 单地址空间，易编程
- 支持动态负载平衡
- 不需要显式数据分配
- 有 Cache 和硬件一致性支持

- 通信延迟低，通常通过 Load/Store 完成通信

15.3 SMP 问题

SMP 的问题包括：

- 可靠性瓶颈：BUS、OS、SM
- CPU 与内存之间通信延迟相对变大
- 处理器越多，竞争越激烈
- 带宽增长慢
- 可扩充性差

因此传统 SMP 很难扩展到很大规模，后续发展方向之一是 CC-NUMA。

16. 大规模并行机 MPP

16.1 MPP 基本特点

MPP 由成百上千个处理器组成。

特点：

- NORMA 结构
- 分布式存储
- 高带宽、低延迟定制互连
- 使用商用处理器
- 通过消息传递编程
- 代表系统包括 Intel Option Red、IBM SP2、Dawning 1000

MPP 的设计目标是可扩充性。

16.2 MPP 系统设计关注点

课件强调 MPP 要考虑：

- 存储器能力
- I/O 能力
- 系统成本
- 通用性和可用性
- 通信要求
- 单一系统映像
- 故障处理

MPP 不只是处理器堆叠，还要平衡 CPU、内存、I/O 和网络。

16.3 典型 MPP 系统比较

第 45 页比较了 Intel/Sandia ASCI Option Red、IBM SP2、SGI/Cray Origin 2000。

比较维度包括：

- 处理器数量
- 峰值性能
- 问世日期
- 处理器类型
- 节点结构
- 内存模型
- 操作系统
- 编程机制

结论是：不同 MPP 可能都叫大规模并行机，但其节点结构、网络、内存模型和编程环境差异很大。

16.4 MPP 使用的 CPU

第 46 页比较 Pentium Pro、PowerPC、Alpha、UltraSPARC、MIPS R10000 等处理器。

指标包括：

- 工艺
- 晶体管数
- 时钟频率
- 电压
- 功耗
- 字长
- Cache
- 执行单元
- 超标量宽度
- 流水线深度

- SPEC 性能

这说明 MPP 的性能不仅取决于网络，也取决于节点处理器能力。

17. SP2 机群型大规模并行机

17.1 SP2 设计策略

IBM SP2 是一种典型机群型大规模并行机。

设计策略包括：

- 机群体系结构
- 标准环境
- 标准编程模型
- 系统可用性
- 精选的单一系统映像

17.2 SP2 系统结构

SP2 使用高性能开关 HPS，多级 Omega 网络。

节点分为：

- 宽节点
- 窄节点
- 窄节点 2

它通过高性能 Omega 网络把多个节点连接起来，同时保留以太网等辅助网络。

SP2 体现了 MPP 和 Cluster 之间的过渡形态。

18. 工作站机群 COW

18.1 COW 基本特点

COW 是 Cluster of Workstations。

特点：

- 分布式存储
- MIMD
- 工作站 + 商用互连网络
- 每个节点是完整计算机
- 每个节点有自己的磁盘和操作系统

与 MPP 不同，MPP 节点通常只有微内核，而 COW 节点运行完整操作系统。

18.2 COW 优点

包括：

- 投资风险小
- 系统结构灵活
- 性能/价格比高
- 能利用分散计算资源
- 可扩充性好

18.3 COW 问题

主要问题是：

- 通信性能不如专用网络
- 并行编程环境复杂

代表系统包括 Berkeley NOW、Alpha Farm、FXCOW。

18.4 典型机群系统

第 49 页列出多个机群研究项目：

- Princeton SHRIMP
- Karlsruhe Parastation
- Rice TreadMarks
- Wisconsin Wind Tunnel
- Globus
- Syracuse WWVM
- HKU Pearl Cluster

- Virginia Legion

这些项目关注方向包括共享虚拟存储、分布并行处理、元计算、广域虚拟机和高性能机群应用。

19. SMP、MPP、机群比较

第 50 页比较三类系统。

19.1 节点数量

- SMP: 通常 $O(10)$ 以下
- MPP: $O(100)$ 到 $O(1000)$
- 机群: 通常 $O(100)$ 以下

19.2 通信方式

- SMP: 共享存储器
- MPP: 消息传递, 或带 DSM 时共享变量
- 机群: 消息传递

19.3 地址空间

- SMP: 单一地址空间
- MPP: 多地址空间, 带 DSM 时也可能是单一地址空间
- 机群: 多个地址空间

19.4 网络

- SMP: 总线或交叉开关
- MPP: 定制网络
- 机群: 商用网络

19.5 性能/价格比

机群通常最高, SMP 和 MPP 一般较低。

所以三者可以这样理解:

SMP: 好编程, 但不易扩展

MPP: 扩展性强, 但成本高、编程复杂

Cluster: 便宜灵活, 但通信性能弱

20. 第三章：并行计算性能评测

第三章讲如何评价并行系统和并行算法的性能。

内容包括:

- 基本性能指标
- 加速比性能定律
- 可扩展性评测标准

21. CPU 基本性能指标

21.1 常见指标

包括:

- 工作负载
- 执行时间
- 浮点运算数
- 指令数目

并行执行时间可以分为:

$$T_n = T_{\text{comput}} + T_{\text{paro}} + T_{\text{comm}}$$

其中:

- T_{comput} : 计算时间
- T_{paro} : 并行开销时间
- T_{comm} : 通信时间

这说明并行程序的运行时间不仅包括计算，还包括通信和同步等额外开销。

21.2 存储器性能

课件第 53 页讲存储层次：

寄存器 → L1 Cache → L2 Cache → 主存 → 磁盘 → 远程存储器

越靠近 CPU：

- 容量越小
- 延迟越低
- 带宽越高

越远离 CPU：

- 容量越大
- 延迟越高
- 带宽越低

例如远程存储器访问可能比本地 Cache 慢很多，这就是并行程序性能下降的重要原因。

21.3 存储器带宽估算

课件用 RISC 指令举例：

```
add r1, r2, r3
```

如果每个寄存器 8 bytes，100 MHz，则一次操作涉及 3 个寄存器：

$$B = 3 \times 8 \times 100 \times 10^6 \text{ B/s} = 2.4 \text{ GB/s}$$

这个例子说明带宽可以从每周期访问的数据量和时钟频率估算。

22. 并行与通信开销

22.1 并行开销很大

课件举例：

PowerPC 每周期 15 ns，可执行 4 flops。

创建一个进程需要 1.4 ms，而这段时间本来可执行 372000 flops。

这说明：

并行管理开销相对于 CPU 计算速度可能非常大。

所以不能随便创建大量细粒度任务。

22.2 Ping-Pong 方法

Ping-Pong 用来测量点到点通信时间。

过程：

1. 节点 0 发送 m 字节消息给节点 1。
2. 节点 1 接收后立即发回节点 0。
3. 节点 0 测量总时间。
4. 总时间除以 2，得到一次单向通信时间。

这是测量通信延迟和带宽的经典方法。

22.3 Hot-Potato 方法

Hot-Potato 也叫 Fire-Brigade。

它是 Ping-Pong 的推广：

0 → 1 → 2 → ... → n-1 → 0

消息沿多个节点传递，用于测量环形或多节点通信性能。

23. 通信开销表达式

23.1 点到点通信

课件给出通信开销：

$$t(m) = t_0 + m / r_{\infty}$$

其中：

- t_0 ：通信启动时间
- m ：消息长度
- r_{∞} ：渐近带宽

23.2 半峰值长度

半峰值长度 $m_{1/2}$ 是达到一半渐近带宽所需的消息长度。

课件给出关系：

$$t_0 = m_{1/2} / r_{\infty} = 1 / \pi_0$$

π_0 表示短消息带宽。

这说明短消息性能主要由启动时间决定，长消息性能主要由带宽决定。

23.3 整体通信

典型整体通信包括：

23.3.1 Broadcast 播送

处理器 0 发送 m 字节给所有处理器。

23.3.2 Gather 收集

处理器 0 接收所有处理器的数据，最终接收 mn 字节。

23.3.3 Scatter 散射

处理器 0 给所有处理器发送不同消息，总发送量为 mn 字节。

23.3.4 Total Exchange 全交换

每个处理器都向其他处理器发送 m 字节，总通信量为 mn^2 。

23.3.5 Circular Shift 循环移位

处理器 i 发送给 $i+1$ ，处理器 $n-1$ 发送给 0，总通信量为 mn 。

24. 机器成本、价格与性价比

24.1 性能/价格比

Performance/Cost Ratio 指单位代价获得的性能。

通常用：

MIPS / million dollars
MFLOPS / million dollars

表示。

24.2 利用率

利用率是可达速度与峰值速度之比。

$$\text{利用率} = \text{实际速度} / \text{峰值速度}$$

峰值性能高不一定代表实际性能高，关键还要看利用率。

25. 算法级性能评测

算法级性能主要看：

- 加速比
- 效率
- 可扩放性

并行系统的加速比是：

对给定应用，并行算法执行速度相对于串行算法加快了多少倍。

课件讲三个加速比定律：

- Amdahl 定律
- Gustafson 定律
- Sun-Ni 定律

26. Amdahl 定律

26.1 基本思想

Amdahl 定律基于固定问题规模。

设：

- P: 处理器数
- W: 总工作负载
- Ws: 串行部分
- Wp: 可并行部分
- $f = W_s / W$: 串行比例
- S: 加速比
- E: 效率

固定负载下，增加处理器只是把可并行部分分摊到多个处理器上。

26.2 加速公式

Amdahl 加速比为：

$$S = p / [1 + f(p - 1)]$$

当 $p \rightarrow \infty$ 时：

$$S \rightarrow 1 / f$$

含义是：

串行部分决定了最大加速上限。

26.3 例子

如果有 1024 个处理器：

$$S_{1024} = 1024 / (1 + 1023f)$$

课件图中：

- $f = 0\%$, $S = 1024$
- $f = 1\%$, $S \approx 91$
- $f = 2\%$, $S \approx 48$
- $f = 3\%$, $S \approx 31$
- $f = 4\%$, $S \approx 24$
- $f = 100\%$, $S = 1$

结论：

即使只有很小串行部分，也会严重限制加速比。

27. Gustafson 定律

27.1 基本思想

Gustafson 定律基于固定运行时间。

它认为在实际大型应用中，增加处理器不是为了把同一个小问题算得更快，而是为了在相同时间内解决更大、更精确的问题。

27.2 加速公式

Gustafson 加速比为：

$$s' = p - f(p - 1)$$

对于 1024 个处理器：

$$s'_{1024} = 1024 - 1023f$$

课件图中，当 $f = 1\%$ 时，加速比约为 1014，而不是 Amdahl 定律中的 91。

27.3 与 Amdahl 的区别

Amdahl：

问题规模固定，处理器越多，时间越短

Gustafson：

时间固定，处理器越多，问题越大

Gustafson 更符合很多实际科学计算场景。

28. Sun 和 Ni 定律

28.1 基本思想

Sun-Ni 定律考虑存储受限问题。

核心观点：

只要存储空间允许，就应尽量增大问题规模，以获得更好、更精确的解。

当处理器数量从 1 增加到 p 时，存储容量也从 M 增加到 pM ，因此可以解决更大问题。

设 $G(p)$ 表示存储容量增加 p 倍时，并行工作负载增加的倍数。

28.2 加速公式

Sun-Ni 的存储受限加速比为：

$$s'' = [f + (1 - f)G(p)] / [f + (1 - f)G(p)/p]$$

28.3 特殊情况

课件给出三个关系：

- 当 $G(p) = 1$ 时，退化为 Amdahl 定律。
- 当 $G(p) = p$ 时，变为 Gustafson 定律。
- 当 $G(p) > p$ 时，Sun-Ni 加速比高于 Amdahl 和 Gustafson。

也就是说，Sun-Ni 是 Amdahl 和 Gustafson 的推广。

29. 加速比讨论

29.1 经验范围

课件给出加速经验公式：

$$p / \log p \leq S \leq p$$

这说明实际加速比通常低于线性加速。

29.2 线性加速比

矩阵相加、内积等通信开销很少的问题，可能接近线性加速。

29.3 $p/\log p$ 加速比

分治类应用问题常出现 $p/\log p$ 级别加速。

29.4 通信密集类问题

通信密集问题的加速比可能被通信成本 $C(p)$ 限制：

$$S = 1 / C(p)$$

29.5 超线性加速

超线性加速是指 $S > p$ 。

可能原因包括：

- 并行后每个节点数据变小，更适合 Cache
- 搜索问题中并行分支提前找到解
- 存储层次行为改善

29.6 绝对加速与相对加速

绝对加速：

最佳并行算法 vs 最佳串行算法

相对加速：

同一算法在单机和并行机上的运行时间比较

30. 可扩放性评测标准

30.1 可扩放性定义

可扩放性是指在特定应用背景下，系统、算法或程序的性能能否随处理器数增加而按比例提高。

影响加速比的因素包括：

- 串行分量
- 通信开销
- 等待
- 竞争
- 冗余操作
- 同步开销
- 并发度不足

30.2 增大问题规模的好处

问题规模增大可能提高加速比，因为：

- 大问题提供更多并发度
- 额外开销增长可能慢于有效计算增长
- 串行部分比例可能随问题规模增大而下降

30.3 可扩放性研究目的

可扩放性研究用于：

- 判断某类问题适合何种并行算法和体系结构
- 预测算法移植到更大机器后的性能
- 为固定问题规模选择最优处理器数
- 指导改进并行算法和并行机结构

课件也指出，目前没有完全统一、标准、普遍接受的严格可扩放性定义。

31. 等效率度量标准

31.1 基本定义

等效率度量关注：

当处理器数 p 增加时，问题规模 W 需要如何增加，才能保持固定效率 E 。

设：

- T_e ：有效计算时间
- T_o ：额外开销时间
- T_p ：并行运行时间
- W ：问题规模，等于最佳串行算法计算量

有：

$$T_e + T_o = p T_p$$

效率为：

$$E = 1 / (1 + T_o / W)$$

如果要保持 E 不变，就必须让 W 随 p 增大。

这个函数称为等效率函数：

$$W = f_E(p)$$

31.2 判断可扩放性

第 71 页图中：

- 曲线 1：很好扩放性
- 曲线 2：可扩放
- 曲线 3：不可扩放

如果维持同样效率时， W 只需缓慢增长，则系统可扩放性好。

如果 W 必须急剧增长，则可扩放性差。

31.3 优缺点

优点：

- 简单
- 可定量计算
- 适合理论分析

缺点：

- 如果 T_o 难以计算，就难以使用
- 在共享存储并行机中，开销常常很复杂

32. 等速度度量标准

32.1 基本定义

设：

- p ：处理器数
- W ：问题规模
- T ：并行执行时间

并行速度：

$$V = W / T$$

平均速度：

$$V' = W / (pT)$$

等速度度量关注：

当处理器数从 p 增大到 p' 时，问题规模 W 应该如何增大，才能保持平均速度不变。

课件给出：

$$\Psi(p, p') = p'W / (pW')$$

32.2 优点

等速度使用机器性能速度指标，比较直观，也容易测量。

32.3 缺点

某些非浮点运算，例如通信、同步、I/O，也会显著影响性能。
因此只看速度可能忽略重要开销。

33. 平均延迟度量标准

33.1 基本定义

平均延迟从更低层次衡量机器性能。

图中每个处理器 P_i 都有：

- 运行时间 T_i
- 延迟 L_i
- 启动前空闲时间
- 结束后空闲时间
- 总并行时间 T_{para}

系统平均延迟反映每个处理器完成分配工作时受到的平均额外开销。

33.2 可扩放性度量

当处理器数由 p 变为 p' ，并保持并行效率不变时，用平均延迟变化来衡量可扩放性。

33.3 优缺点

优点：

- 能在更低层次衡量机器性能
- 能反映通信、等待、同步等细节

缺点：

- 需要特定软硬件支持
- 测量成本较高

34. 本文件总结

PC1-3.pdf 的核心可以概括为三句话：

34.1 第一章讲“结构模型”

包括：

互连网络 → 静态/动态网络 → 标准网络 → PVP/SMP/MPP/DSM/COW → UMA/NUMA/COMA/NORMA

重点是理解并行机如何组织处理器、内存和网络。

34.2 第二章讲“当代系统”

包括：

SMP → MPP → SP2 → COW → 三者比较

重点是理解不同系统的优缺点：

SMP 易编程但扩展差
MPP 扩展强但成本高
Cluster 性价比高但通信弱

34.3 第三章讲“性能评测”

包括：

CPU/存储器/通信指标 → Ping-Pong → Amdahl → Gustafson → Sun-Ni → 可扩展性

重点是理解并行程序为什么不能只看处理器数，还必须看：

- 串行比例
- 通信开销
- 同步开销
- 问题规模
- 存储容量
- 系统可扩展性

整份课件的主线是：

并行计算需要硬件结构支撑；
硬件结构的关键是互连和存储模型；
系统性能不仅取决于峰值速度，还取决于通信、同步、负载均衡和可扩展性。